

SMART ROOM INTELLIGENCE SYSTEM

A Fully Local Edge IoT Security & Monitoring Platform

Technical Project Report

For educational purpose only

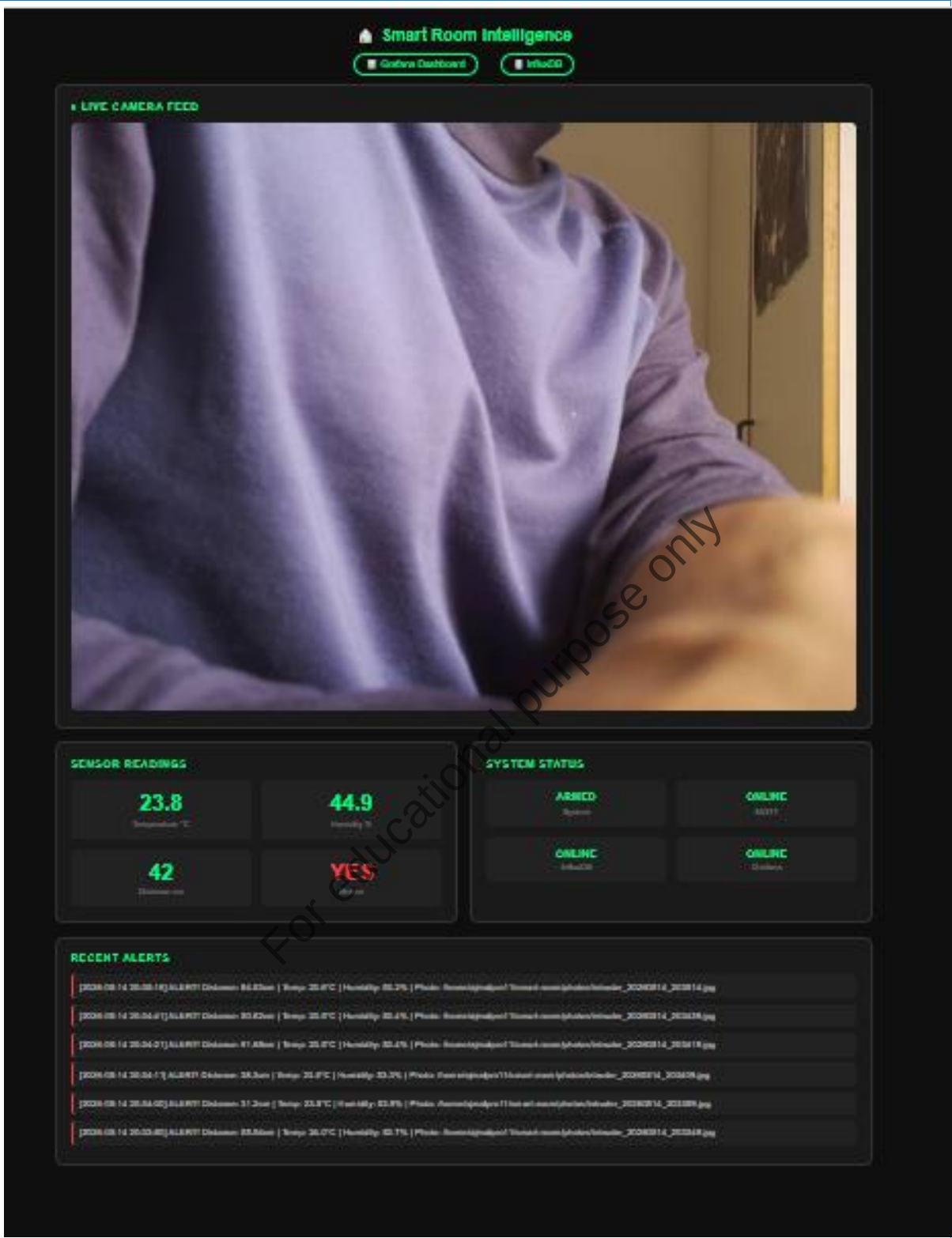


Figure 1 — Smart Room Intelligence Dashboard running live on Raspberry Pi 5

Author

Ajuma! Shamsudeen

TH Rosenheim, Germany

github.com/ajmalpro11/smart-room-intelligence

May 2026

For educational purpose only

Abstract

This report documents the complete design, development, and deployment of the Smart Room Intelligence System — a fully local, edge-based IoT security and environment monitoring platform built on Raspberry Pi 5. The system integrates four physical sensors (HC-SR04 ultrasonic distance sensor, IR obstacle sensor, DHT22 temperature and humidity sensor, and Pi Camera Module 3) with a passive buzzer to create an intelligent intruder detection system.

When motion is detected and an object is confirmed within range, the system automatically captures a timestamped photograph, sounds an audio alert, and logs the event with environmental context. All sensor data is continuously published via MQTT messaging protocol to a local InfluxDB time-series database and visualized in real-time through a Grafana dashboard. A Flask web application serves a live camera feed, current sensor readings, and recent alert history accessible from any device on the local network.

The entire system runs locally on the Raspberry Pi 5 without any cloud dependency, containerised using Docker Compose. The project demonstrates the integration of hardware interfacing, embedded Python programming, IoT communication protocols, time-series data storage, and web development into a single cohesive system. All code, documentation, and 27 development progress screenshots are publicly available on GitHub.

For educational purpose only

Table of Contents

1. Introduction	4
2. System Overview	5
3. Hardware Components	6
4. Software Architecture	11
5. Python Libraries	13
6. Implementation — Phase by Phase	17
6.1 Phase 1 — Environment Setup	17
6.2 Phase 1 — Sensor Integration	19
6.3 Phase 2 — Decision Engine	25
6.4 Phase 3 — IoT Data Pipeline	27
6.5 Phase 4 — Web Dashboard	31
7. Step-by-Step Replication Guide	34
8. Challenges and Solutions	44
9. Results and Discussion	46
10. Future Work	48
11. Conclusion	49
Appendix A — GPIO Pin Reference	50
Appendix B — Complete Library Reference	51
Appendix C — Docker Services Reference	53
Appendix D — API Endpoints	54
Appendix E — Troubleshooting Guide	55

1. Introduction

The Internet of Things (IoT) has transformed how we interact with physical environments. Sensors, microcontrollers, and communication protocols now allow everyday objects to generate, transmit, and act on real-world data. However, most commercial IoT systems rely on cloud infrastructure — raising concerns about privacy, latency, internet dependency, and ongoing subscription costs.

This project was conceived as a hands-on exploration of edge IoT — processing all data locally on the device itself, with no data leaving the premises. Built on the Raspberry Pi 5, the most capable single-board computer in the Pi family, the Smart Room Intelligence System demonstrates that a complete, professional-grade IoT platform can be built for approximately 50 euros using open-source tools and components available from any electronics retailer in Germany.

The project was developed iteratively over multiple sessions, with each phase building on the last. Every decision — hardware selection, library choice, protocol design, dashboard architecture — is documented in this report so that any student or developer can replicate or extend the system.

1.1 Project Objectives

- Integrate multiple sensors into a unified data collection system on Raspberry Pi 5
- Implement intelligent multi-sensor fusion for reliable intruder detection
- Build a complete IoT data pipeline from sensor reading to live dashboard
- Serve a web dashboard accessible from any device on the local network
- Run everything locally with no cloud dependency, subscription, or internet requirement
- Document the complete development process for full reproducibility
- Apply professional software practices: Git version control, Docker containerisation, secure credential management

1.2 Scope

The system covers hardware wiring, embedded Python programming, MQTT messaging, time-series data storage, data visualisation, and web development. Machine learning anomaly detection and stepper motor pan-tilt camera control are identified as future extensions but are outside the scope of this report.

1.3 GitHub Repository

All source code, documentation, and progress screenshots are publicly available at:

<https://github.com/ajmalpro11/smart-room-intelligence>

2. System Overview

The Smart Room Intelligence System is composed of five functional layers, each building on the previous:

Layer	Components	Purpose
Physical Layer	HC-SR04, IR sensor, DHT22, Pi Camera, Buzzer	Real-world data collection
Application Layer	Python scripts (gpiozero, adafruit, picamera2)	Sensor reading and control
Messaging Layer	Mosquitto MQTT Broker	Reliable data transport between components
Storage Layer	InfluxDB time-series database	Persistent sensor data storage
Presentation Layer	Grafana + Flask web app	Visualisation and live monitoring

2.1 Data Flow

1. publisher.py reads all sensors every 2 seconds
2. Sensor readings are serialised as JSON and published to MQTT topic smartroom/sensors
3. subscriber.py receives the MQTT message and writes each field to InfluxDB
4. Grafana queries InfluxDB continuously and renders live charts
5. Flask app.py subscribes to the same MQTT topic and serves data to the browser
6. Browser polls /sensors endpoint every 2 seconds and updates the dashboard
7. Simultaneously, decision_engine.py monitors for intrusion conditions
8. On intrusion: buzzer sounds, camera captures photo, alert is logged to file

2.2 Key Design Principles

- Edge-first: all processing on the Raspberry Pi, zero cloud dependency
- Separation of concerns: each script has exactly one responsibility
- Fault tolerance: sensor errors caught gracefully, system continues running
- Security: credentials in .env file, never committed to version control
- Reproducibility: Docker Compose ensures identical environment on any machine

3. Hardware Components

3.1 Raspberry Pi 5

The Raspberry Pi 5 is the main computing platform. With a 2.4GHz quad-core ARM Cortex-A76 processor and 4GB LPDDR4X RAM, it is significantly more powerful than previous Pi generations — capable of running Docker containers, a web server, database, and multiple sensor scripts simultaneously.

Specification	Value
Processor	Broadcom BCM2712, quad-core ARM Cortex-A76 @ 2.4GHz
RAM	4GB LPDDR4X-4267 SDRAM
Storage	64GB microSD card
GPIO	40-pin header (26 usable GPIO pins)
Camera Interface	2x MIPI CSI-2 connectors (CAM0, CAM1)
OS	Debian GNU/Linux 13 (Trixie)
Python Version	3.13

3.2 HC-SR04 Ultrasonic Distance Sensor

The HC-SR04 measures distance by emitting a 40kHz ultrasonic pulse and measuring the time for the echo to return. It has a range of 2cm to 400cm with approximately 3mm accuracy. It works like a bat's echolocation — sending out a sound pulse you cannot hear, bouncing it off an object, and calculating distance from the round-trip time.

Pin	Function	Connected to
VCC	Power supply (5V)	Pi Pin 2 (5V)
TRIG	Trigger pulse input	Pi Pin 12 (GPIO 18)
ECHO	Echo output — requires voltage divider	1kΩ+2kΩ divider → Pi Pin 18 (GPIO 24)
GND	Ground	Pi Pin 6 (GND)

Voltage Divider — Critical Safety Component

The HC-SR04 ECHO pin outputs 5V, which exceeds the Raspberry Pi GPIO maximum of 3.3V. Direct connection would damage the Pi permanently. A voltage divider using two resistors reduces the signal safely:

$$V_{\text{out}} = V_{\text{in}} \times R_2 / (R_1 + R_2) = 5V \times 2000 / (1000 + 2000) = 3.33V \quad \checkmark$$

Use a 1kΩ resistor (Brown-Black-Red-Gold) in series from ECHO, and a 2kΩ resistor (Red-Black-Red-Gold) from the junction to GND. The GPIO pin connects at the junction.

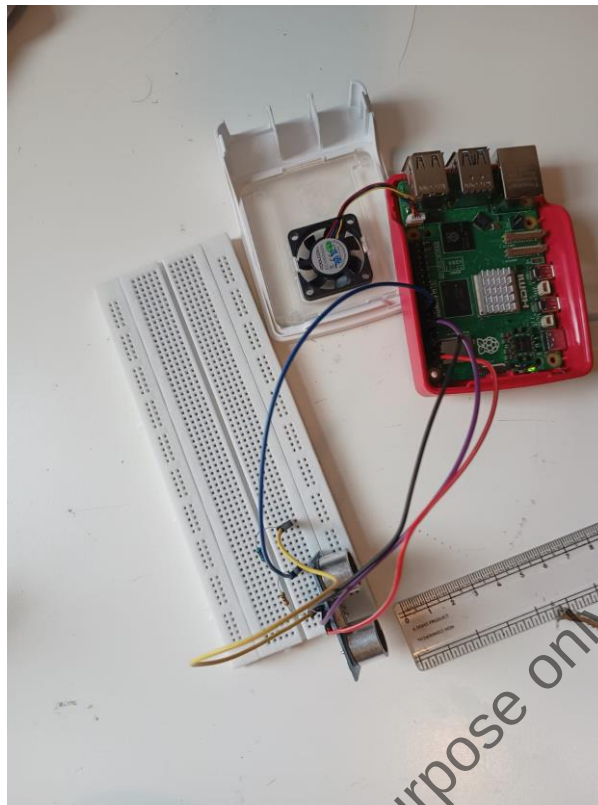


Figure 2 — HC-SR04 ultrasonic sensor wired to breadboard with voltage divider

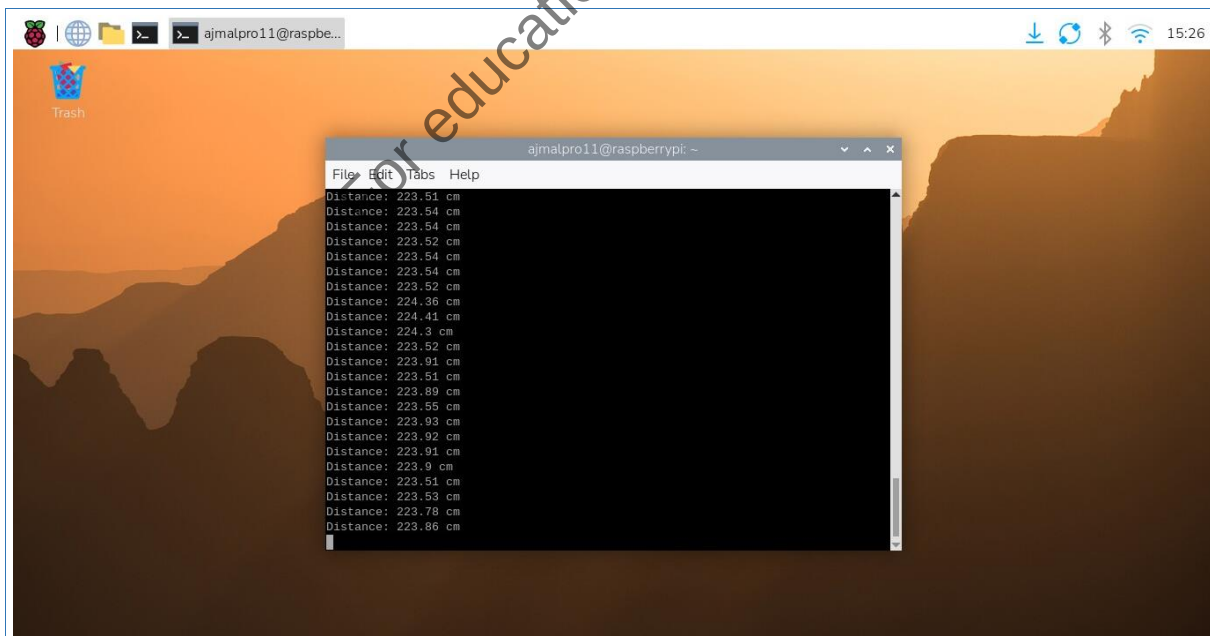


Figure 3 — Live distance readings from HC-SR04 in terminal

3.3 IR Obstacle Sensor

The infrared obstacle sensor emits IR light and detects objects by measuring reflected IR radiation. It includes a potentiometer for sensitivity adjustment and outputs a digital HIGH/LOW signal. Detection range is approximately 2cm to 30cm depending on potentiometer calibration. The sensor requires careful calibration — the sweet spot is found by slowly turning the potentiometer until the detection LED changes state as an object enters the detection zone.

Pin	Function	Connected to
VCC (+)	Power supply (3.3V)	Pi Pin 1 (3.3V)
GND (-)	Ground	Pi Pin 9 (GND)
OUT	Digital output signal	Pi Pin 16 (GPIO 23)



Figure 4 — IR obstacle sensor wired to Raspberry Pi 5

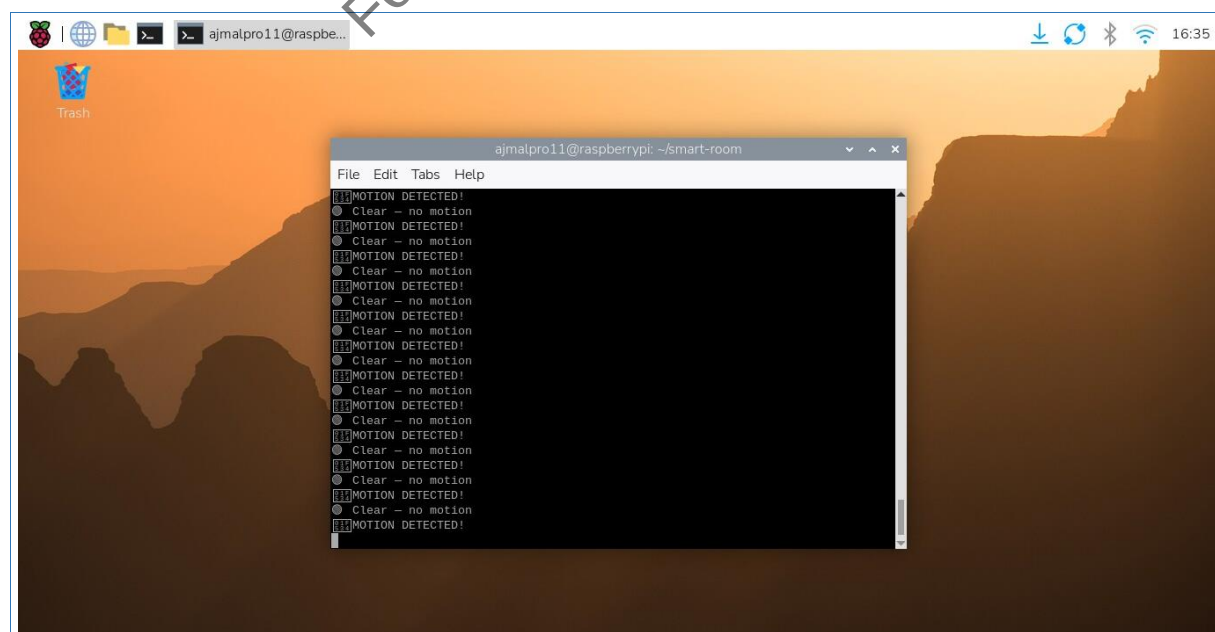


Figure 5 — IR sensor detecting and clearing motion in real time

3.4 DHT22 Temperature and Humidity Sensor

The DHT22 (AM2302) provides calibrated temperature (-40°C to $+80^{\circ}\text{C}$, $\pm 0.5^{\circ}\text{C}$) and relative humidity (0-100%, $\pm 2-5\%$) readings. The PCB-mounted version used includes a built-in pull-up resistor, simplifying wiring significantly to just three wires.

Pin	Function	Connected to
+ (VCC)	Power supply (3.3V)	Pi Pin 1 (3.3V)
out (DATA)	Single-wire digital data	Pi Pin 11 (GPIO 17)
- (GND)	Ground	Pi Pin 9 (GND)

Important constraint: The DHT22 hardware requires a minimum of 2 seconds between consecutive readings. All scripts respect this limitation — attempting to read faster causes `RuntimeError` exceptions.

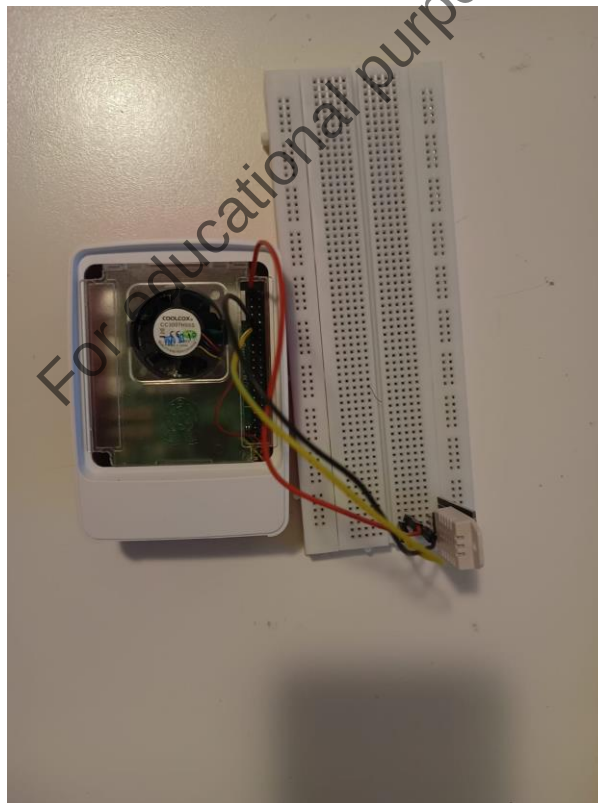


Figure 6 — DHT22 PCB sensor wired to Raspberry Pi 5

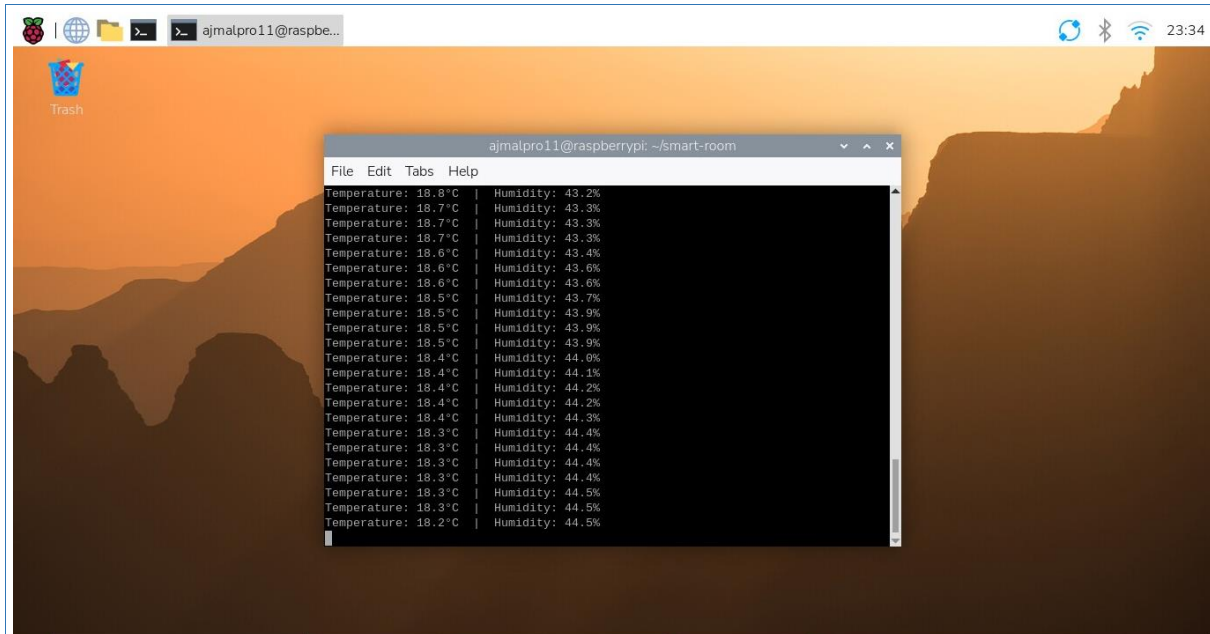


Figure 7 — Live temperature and humidity readings from DHT22

3.5 Pi Camera Module 3

The Pi Camera Module 3 uses the Sony IMX708 12-megapixel sensor with autofocus capability. It connects via CSI ribbon cable to the CAM0 connector. On Raspberry Pi 5 with Debian Trixie, the correct cable orientation has the blue plastic side facing away from the USB ports.

Specification	Value
Sensor	Sony IMX708
Resolution	12 megapixels (4608 x 2592)
Still capture resolution used	1920 x 1080 (Full HD)
Video stream resolution used	640 x 480
Connection	CSI ribbon cable to CAM0
Cable orientation (Pi 5)	Blue side facing away from USB ports

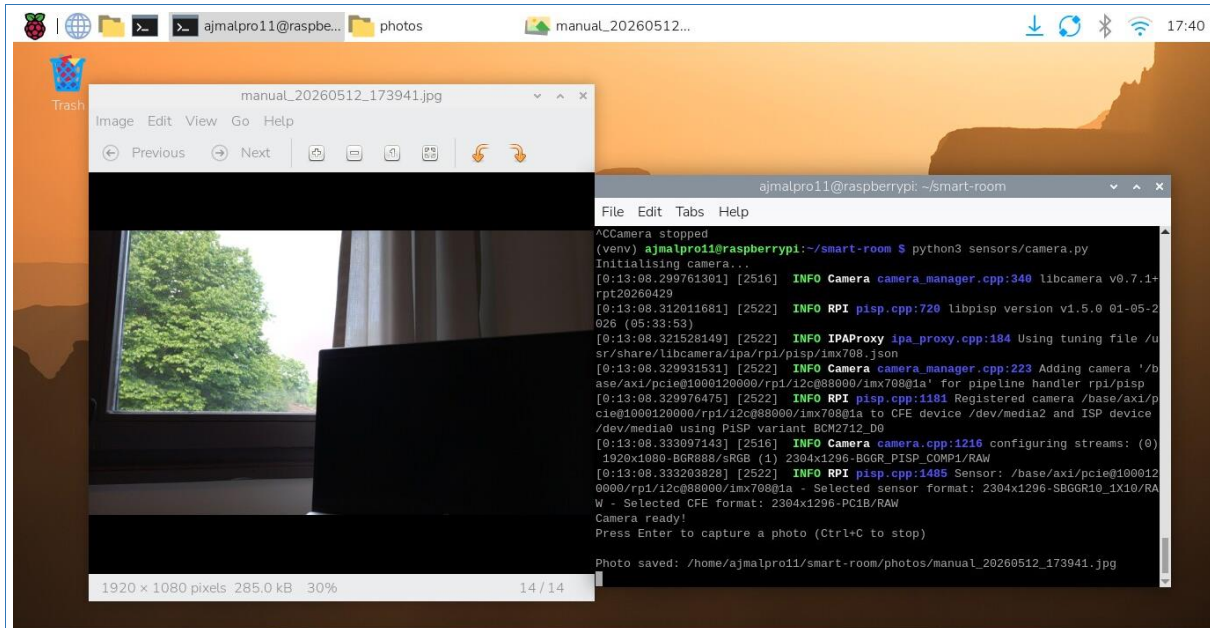


Figure 8 — Pi Camera Module 3 successfully detected and capturing photo

3.6 Passive Buzzer

A passive buzzer produces sound when driven with a signal. Used as the audio alert component — sounds 3 short beeps when an intrusion is detected.

Pin	Function	Connected to
+ (positive)	Signal input	Pi Pin 22 (GPIO 25)
- (negative)	Ground	Pi Pin 20 (GND)

3.7 Complete Wiring

The image below shows all sensors wired together on the breadboard — the complete hardware assembly used in this project.

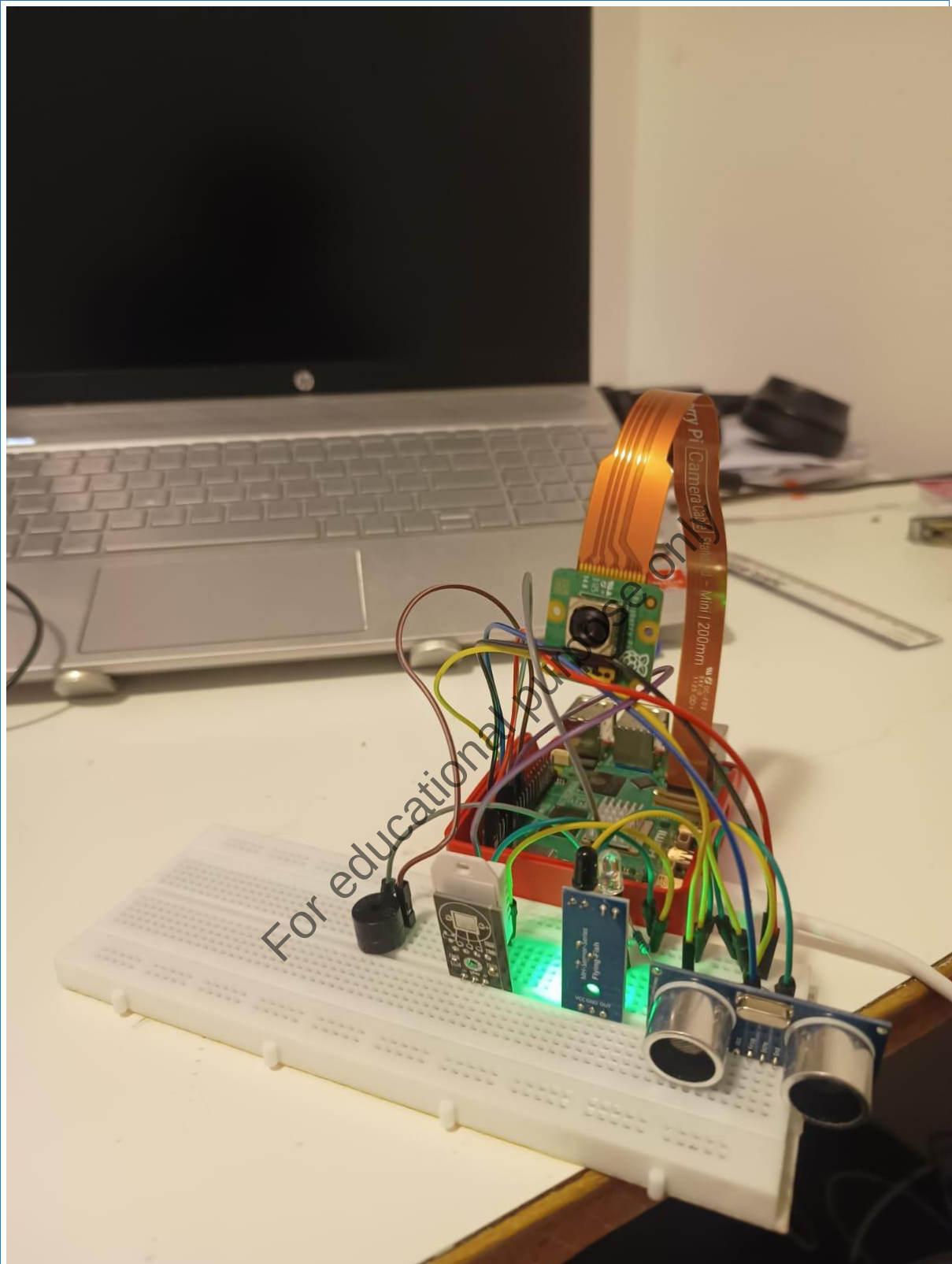


Figure 9 — Complete hardware assembly: all sensors wired together on breadboard

4. Software Architecture

4.1 Project Structure

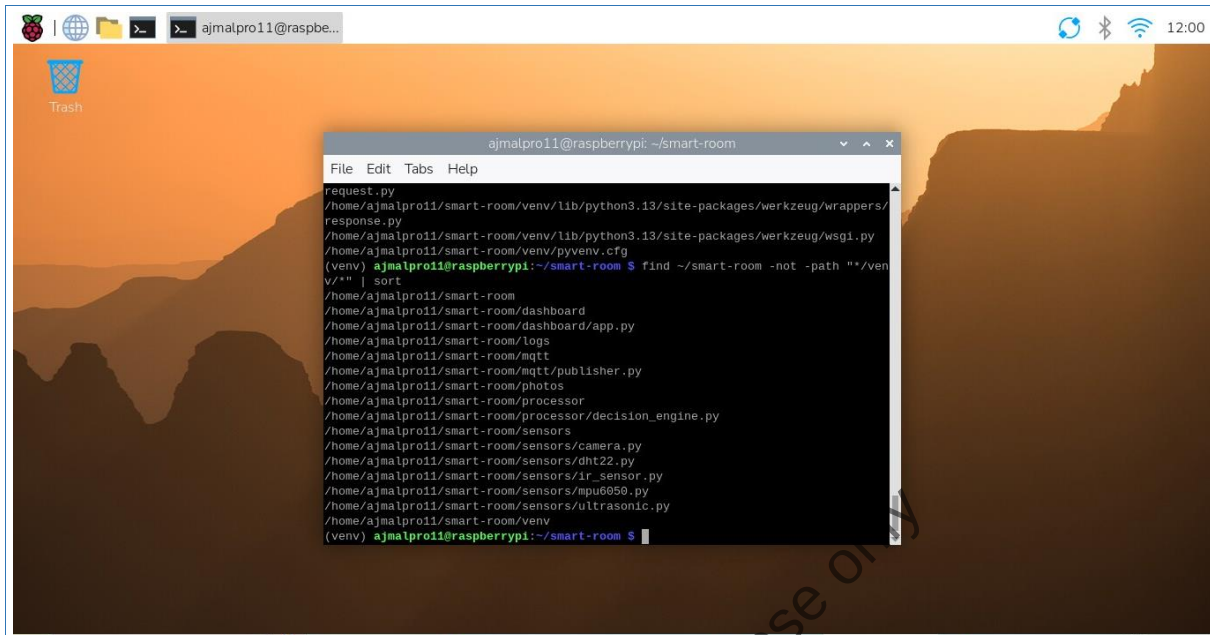


Figure 10 — Project folder structure showing all scripts and configuration files

```

smart-room/
├── sensors/           ← Individual sensor test scripts
├── mqtt/             ← Publisher (sensors→MQTT) and Subscriber
                        (MQTT→InfluxDB)
├── processor/        ← Decision engine (multi-sensor fusion + alerts)
├── dashboard/        ← Flask web server + HTML template
├── mosquitto/config/ ← MQTT broker configuration
├── grafana/dashboards/ ← Auto-provisioned dashboard JSON
├── docker-compose.yml ← Full stack container orchestration
├── start.sh          ← Single command to start everything
├── stop.sh           ← Single command to stop everything
└── .env              ← Credentials (excluded from Git)

```

4.2 MQTT Message Format

A single MQTT topic carries all sensor data as a JSON payload published every 2 seconds:

```

Topic: smartroom/sensors
{
  "timestamp": "2026-05-14T20:34:21",
  "distance": 45.23,
  "motion": 1,
  "temperature": 23.5,
  "humidity": 51.4
}

```

4.3 Docker Services

Service	Image	Port	Purpose
mosquitto	eclipse-mosquitto:latest	1883	MQTT message broker
influxdb	influxdb:2.7	8086	Time-series database
grafana	grafana/grafana:latest	3000	Data visualisation

For educational purpose only

5. Python Libraries

All libraries were installed inside a Python virtual environment created with `--system-site-packages` access. This flag is essential on Raspberry Pi 5 with Debian Trixie to allow access to system-level libcamera bindings that cannot be installed via pip.

```
python3 -m venv venv --system-site-packages
source venv/bin/activate
```

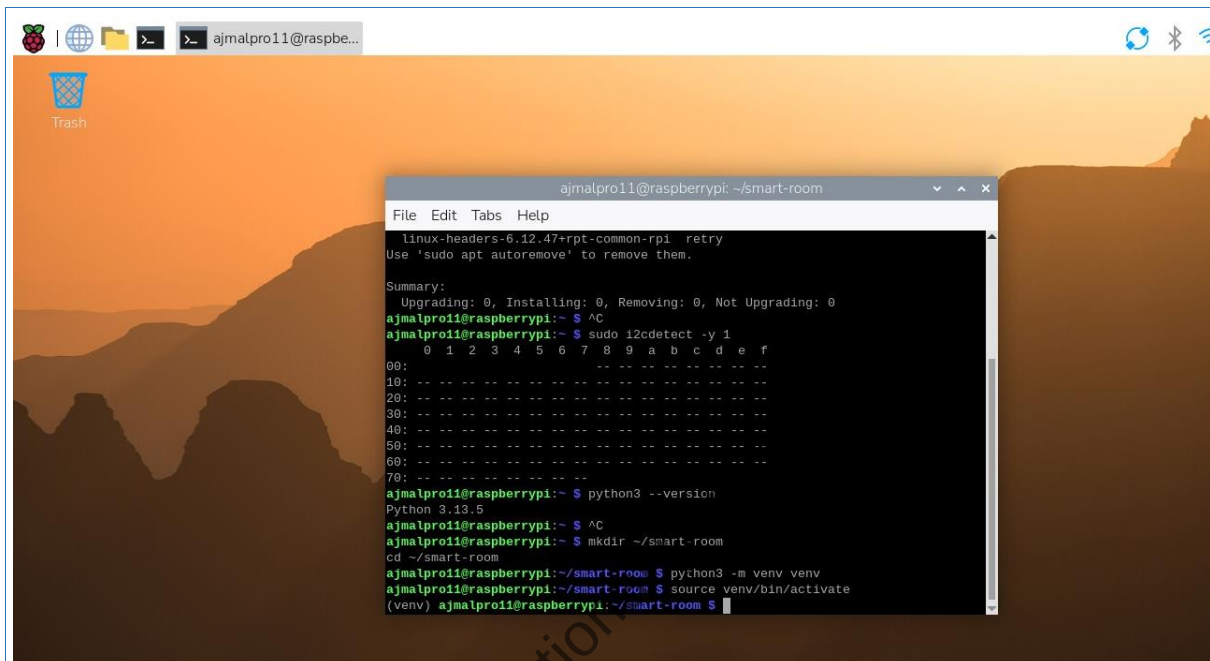


Figure 11 — Virtual environment created and activated showing (venv) prefix

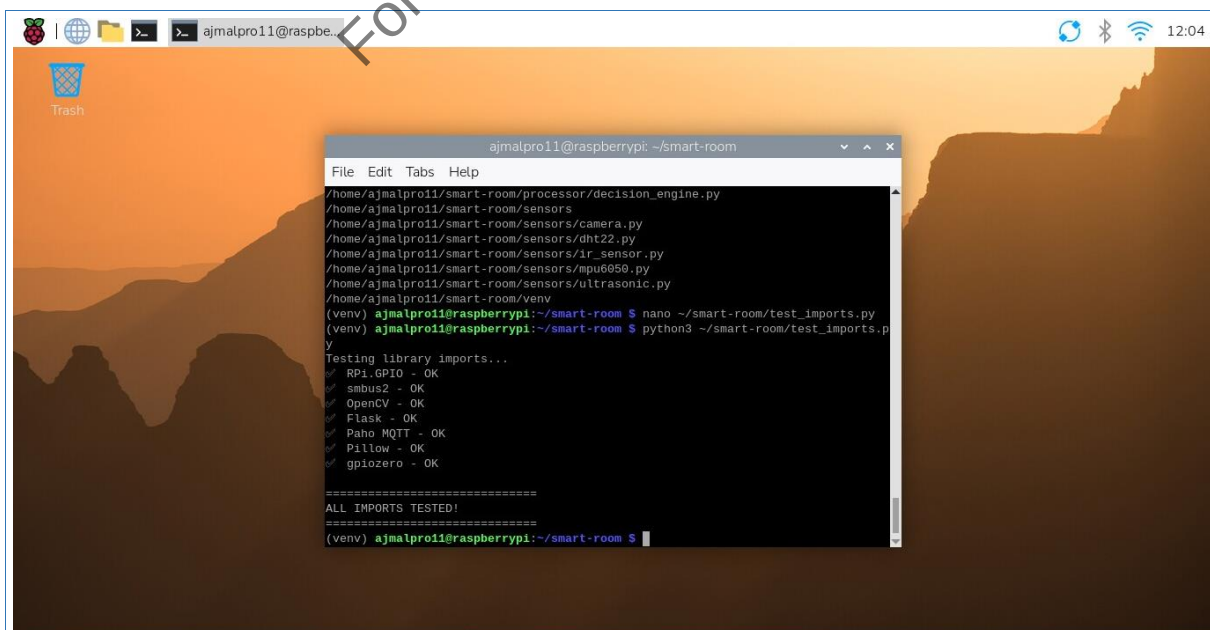


Figure 12 — All libraries successfully imported and verified

5.1 GPIO and Hardware Control

gpiozero (2.0.1)

The primary GPIO control library providing high-level abstractions. On Raspberry Pi 5, gpiozero requires the lgpio backend due to the new RP1 chip architecture. The environment variable must be set before any gpiozero import:

```
export GPIOZERO_PIN_FACTORY=lgpio
```

Key classes used: DistanceSensor for HC-SR04 (returns distance in metres), DigitalInputDevice for IR sensor (returns 0 or 1), Buzzer for audio alerts.

lgpio (0.2.2.0)

The GPIO backend library specifically designed for Raspberry Pi 5. The older RPi.GPIO library fails with 'Cannot determine SOC peripheral base address' on Pi 5. lgpio requires system build dependencies before pip installation:

```
sudo apt install liblgpio-dev swig -y
pip install lgpio
```

smbus2 (0.6.1)

Implements I2C protocol in Python for communicating with I2C devices such as the MPU-6050 accelerometer (pending soldering).

5.2 Sensor Libraries

adafruit-circuitpython-dht (4.0.12)

Adafruit's official DHT22 library. Uses the board module for clean GPIO pin references. RuntimeError exceptions (occurring on ~1 in 20 readings due to signal timing) must be caught gracefully:

```
import adafruit_dht, board
sensor = adafruit_dht.DHT22(board.D17)
try:
    temp = sensor.temperature
except RuntimeError:
    pass # retry next cycle
```

picamera2 (0.3.36)

Official Python library for Pi cameras on Pi 4 and Pi 5. Replaces the older picamera library. Provides still capture and video streaming modes. Requires system-site-packages access for libcamera bindings.

5.3 Image Processing

opencv-python (4.13.0)

Used for JPEG encoding of camera frames for streaming, and image flipping to correct physical mounting orientation. The camera image was inverted due to mounting — corrected with:

```
|frame = cv2.flip(frame, -1)    # flip both axes = 180 degree rotation
```

5.4 Web and Networking

Flask (3.1.3)

Lightweight Python web framework serving the dashboard. Four routes: / (HTML page), /video_feed (MJPEG stream), /sensors (JSON data), /alerts (JSON log).

paho-mqtt (2.1.0)

Eclipse Paho MQTT client. Used in publisher.py for publishing and in both subscriber.py and app.py for subscribing to sensor data.

5.5 Data Storage

influxdb-client (1.50.0)

Official InfluxDB 2.x Python client. Data written as Point objects with measurement name, location tag, and field values. Uses Flux query language for dashboard queries.

5.6 Security

python-dotenv

Loads credentials from .env file into environment variables. The standard industry approach for keeping secrets out of source code. The .env file is excluded from Git via .gitignore.

6. Implementation — Phase by Phase

6.1 Phase 1 — Environment Setup

Development environment preparation was completed before any hardware work. This ensures a clean, reproducible foundation.

System Update and Interface Configuration

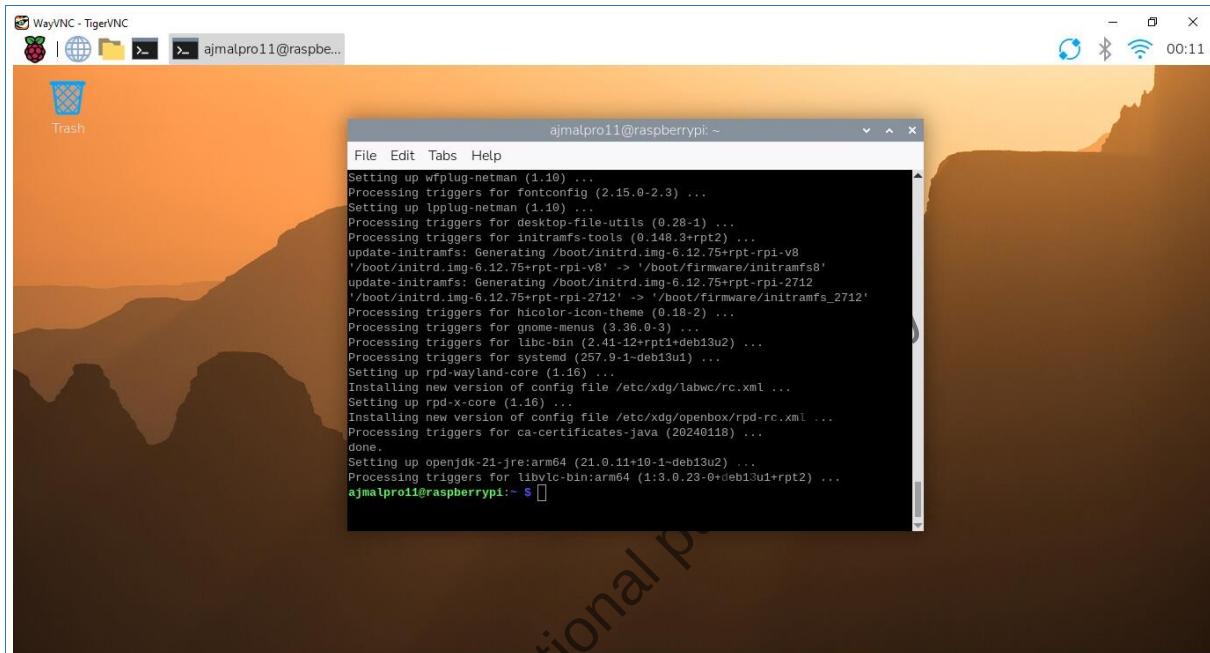


Figure 13 — System updated and ready for development

I2C Verification

Three I2C buses were detected on the Pi 5: /dev/i2c-1 (main user bus used for sensors), /dev/i2c-13 and /dev/i2c-14 (internal system buses, not used). The i2cdetect tool confirms the bus is operational:

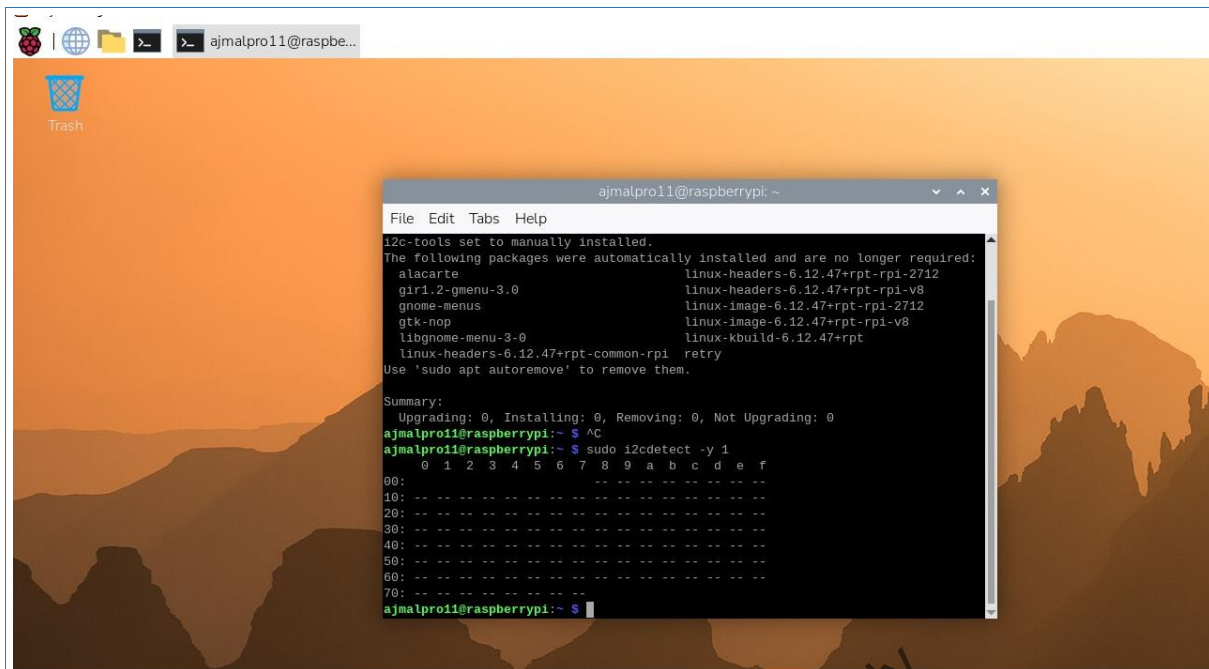


Figure 14 — I2C scan showing empty grid before sensor connection (all dashes expected)

GitHub Setup

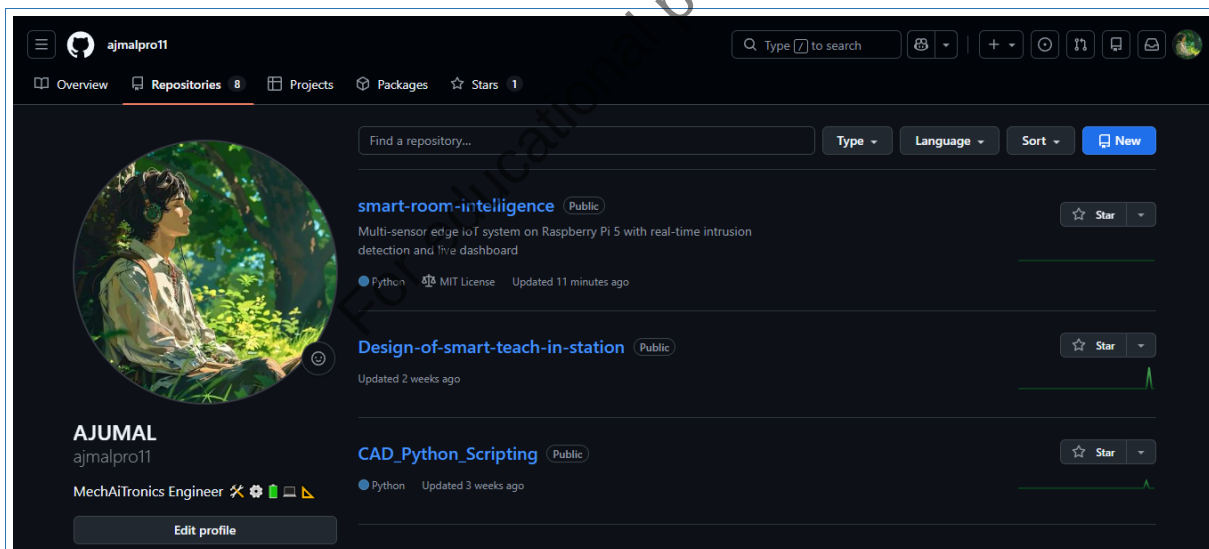


Figure 15 — Project live on GitHub with initial commit

GitHub README

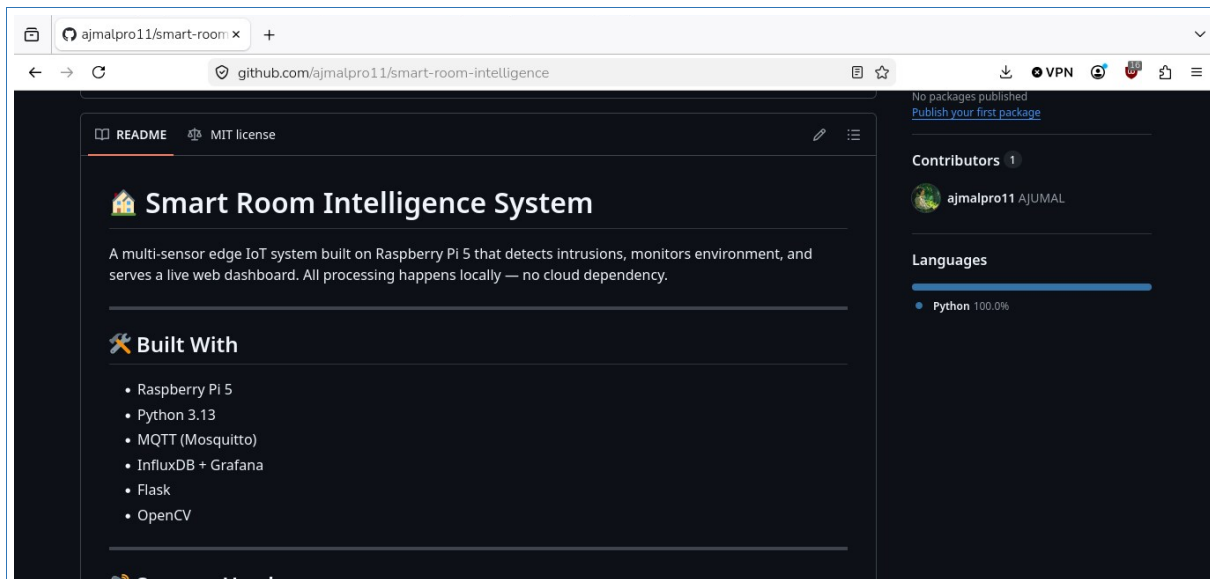


Figure 16 — Professional README on GitHub showing project documentation

6.2 Phase 1 — Sensor Integration

Each sensor was wired and tested individually before integration. This methodical approach isolates problems early.

HC-SR04 Ultrasonic Sensor

Initial attempts using `RPi.GPIO` failed with 'Cannot determine SOC peripheral base address' — a known Pi 5 incompatibility. The solution was switching to `gpiozero` with `Igpio` backend. The `DistanceSensor` class encapsulates the complete pulse-echo timing:

```
sensor = DistanceSensor(echo=24, trigger=18, max_distance=4)
distance_cm = sensor.distance * 100
```

IR Obstacle Sensor

The sensor required potentiometer calibration to find the detection sweet spot. Setting `pull_up=True` enables the Pi's internal pull-up resistor for a stable signal.

DHT22 Temperature and Humidity

The PCB module's built-in pull-up resistor eliminates external components. `RuntimeError` handling is essential for production reliability:

Pi Camera Module 3

Camera required two fixes: `--system-site-packages` for `libcamera` bindings, and `cv2.flip(-1)` for correct image orientation. Both are thoroughly documented for reproducibility.

Decision Engine

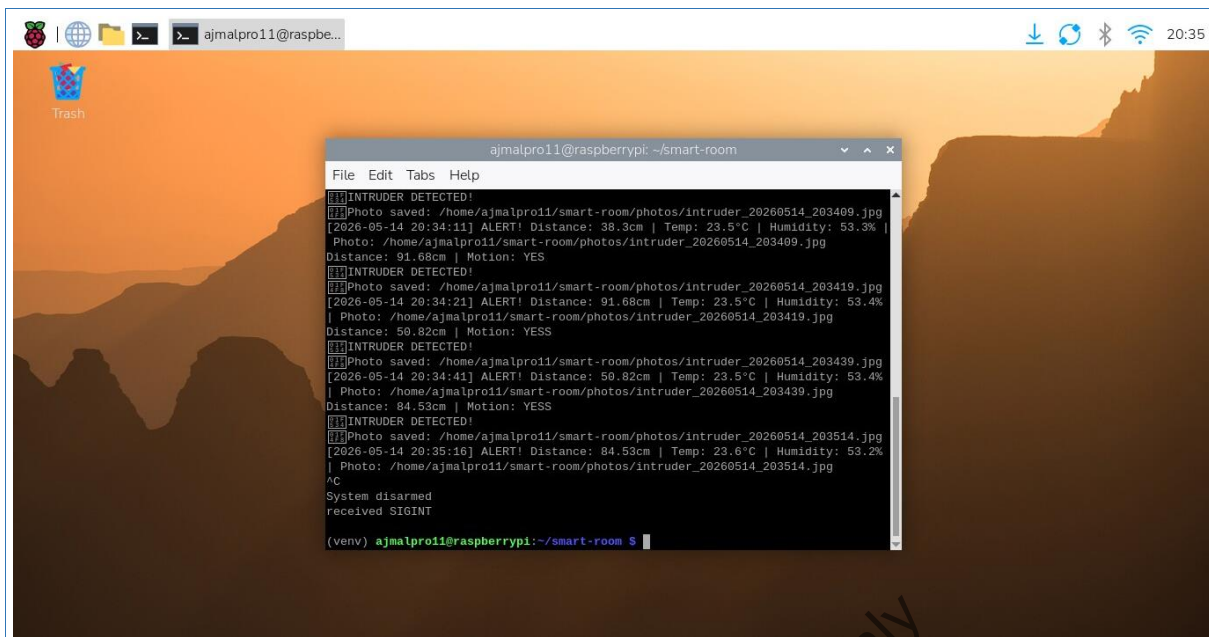


Figure 17 — Decision engine running: multi-sensor fusion triggering live alerts with distance, temperature, and humidity logged

An auto-captured intruder photo saved to the photos folder with timestamp in the filename:

6.3 Phase 2 — Decision Engine

The decision engine applies multi-sensor fusion logic. By requiring both IR and ultrasonic to agree simultaneously, false positive rates are dramatically reduced compared to single-sensor detection:

```

if motion_detected AND distance < 150cm:
    if time_since_last_alert > 10 seconds: # cooldown
        capture_photo()           # timestamped JPEG
        sound_alert()             # 3 short beeps
        log_alert(distance, temperature, humidity, photo_path)

```

The 10-second cooldown prevents alert flooding — without it, a person standing in the detection zone would generate dozens of alerts per minute.

6.4 Phase 3 — IoT Data Pipeline

Docker Stack

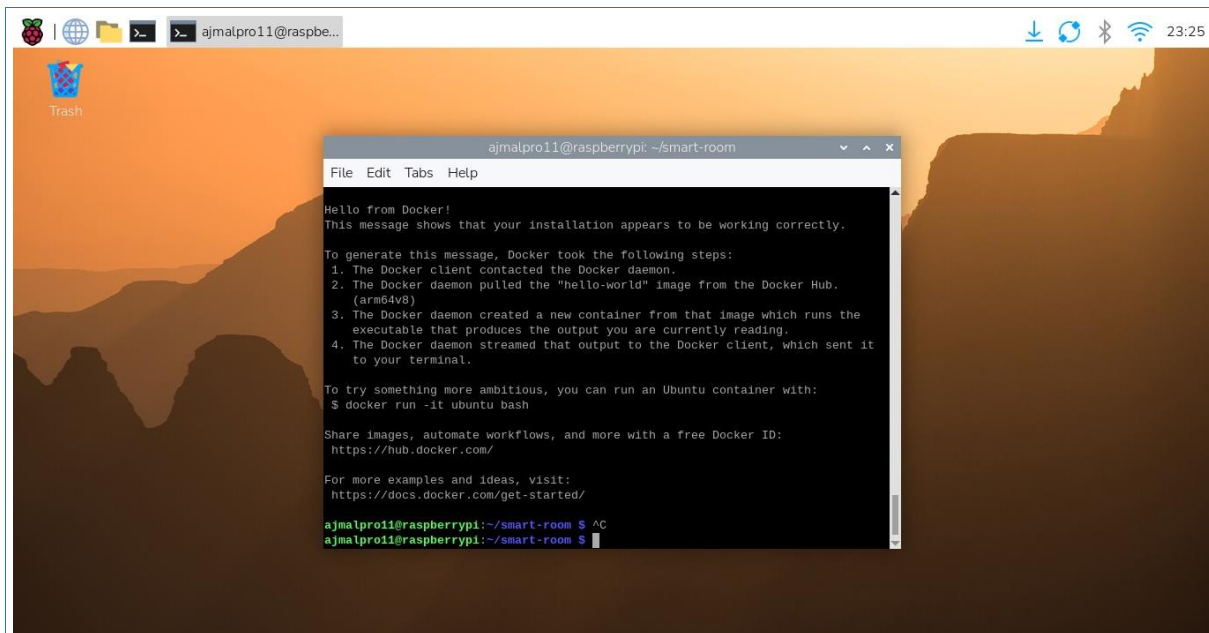


Figure 18 — Docker installed and running on Raspberry Pi 5

MQTT Publisher and Subscriber

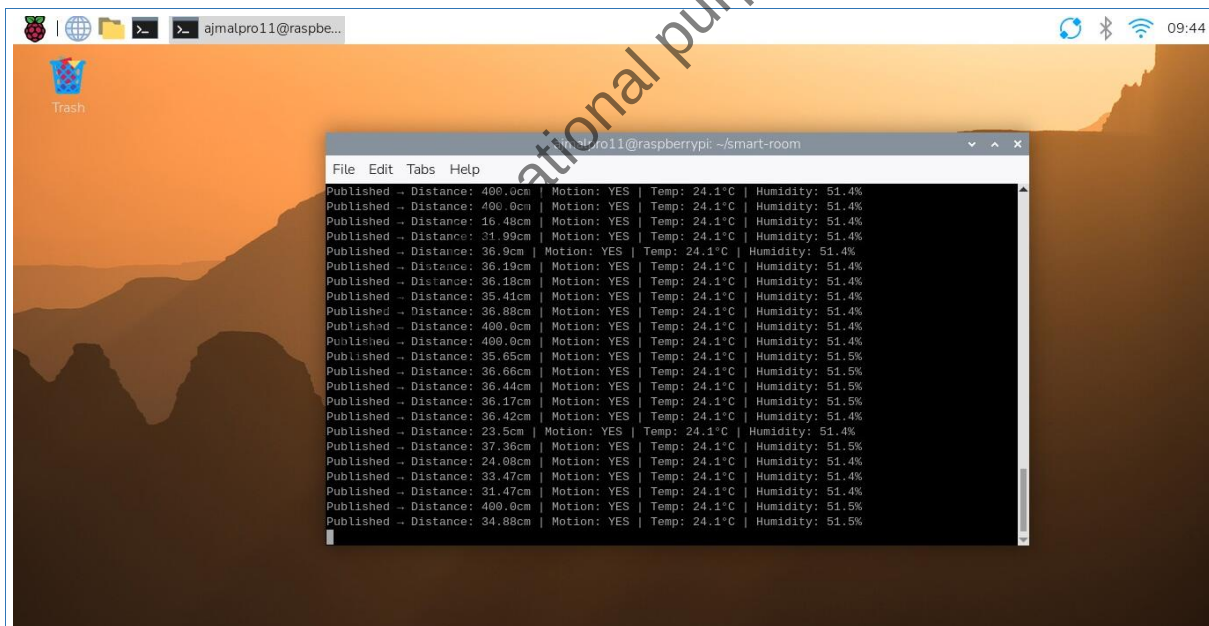


Figure 19 — MQTT publisher streaming live sensor data every 2 seconds

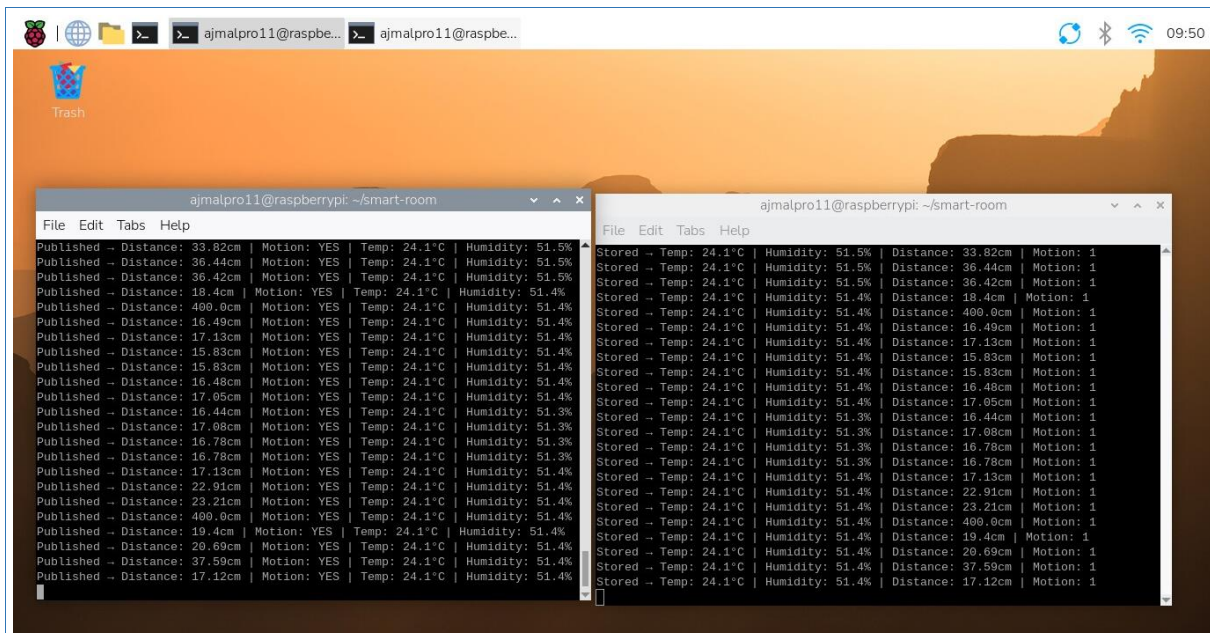


Figure 20 — Full MQTT pipeline: publisher (left) and subscriber storing to InfluxDB (right)

Grafana Dashboard

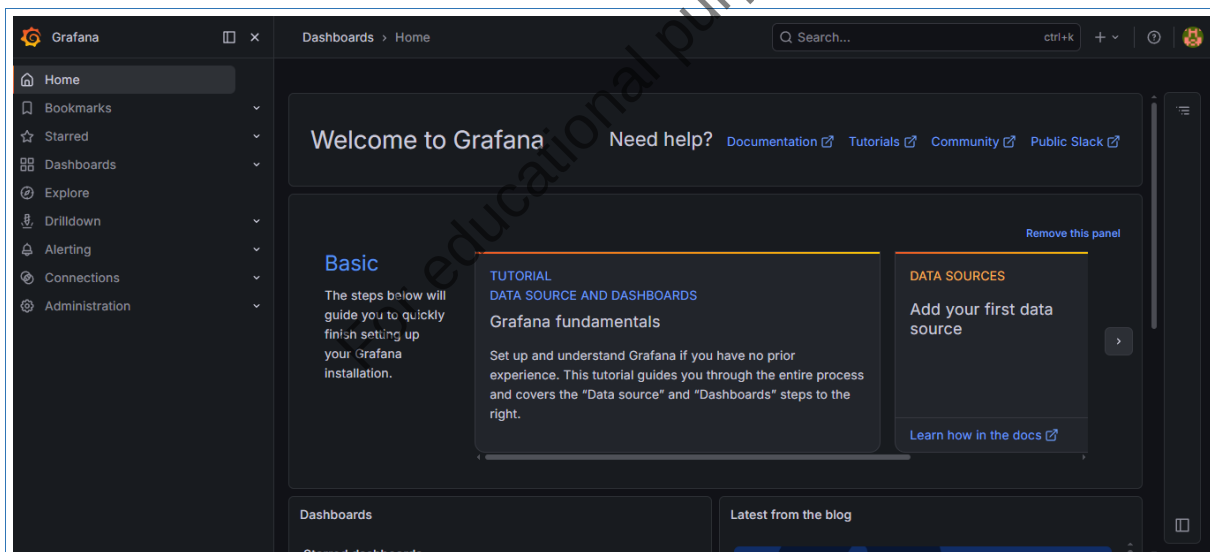


Figure 21 — Grafana running and accessible on the local network

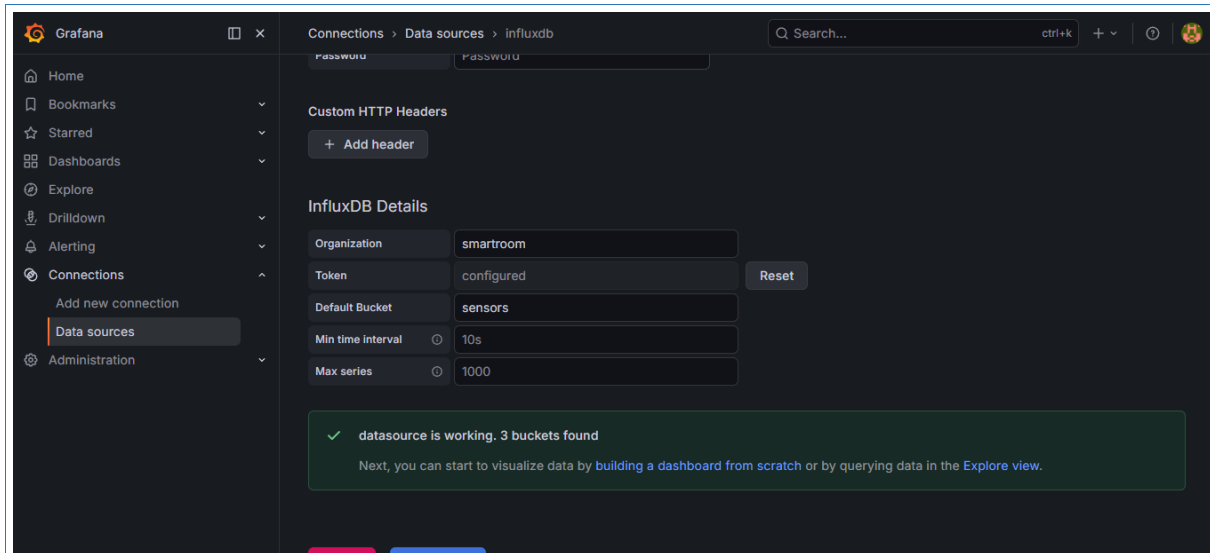


Figure 22 — InfluxDB successfully connected to Grafana as data source

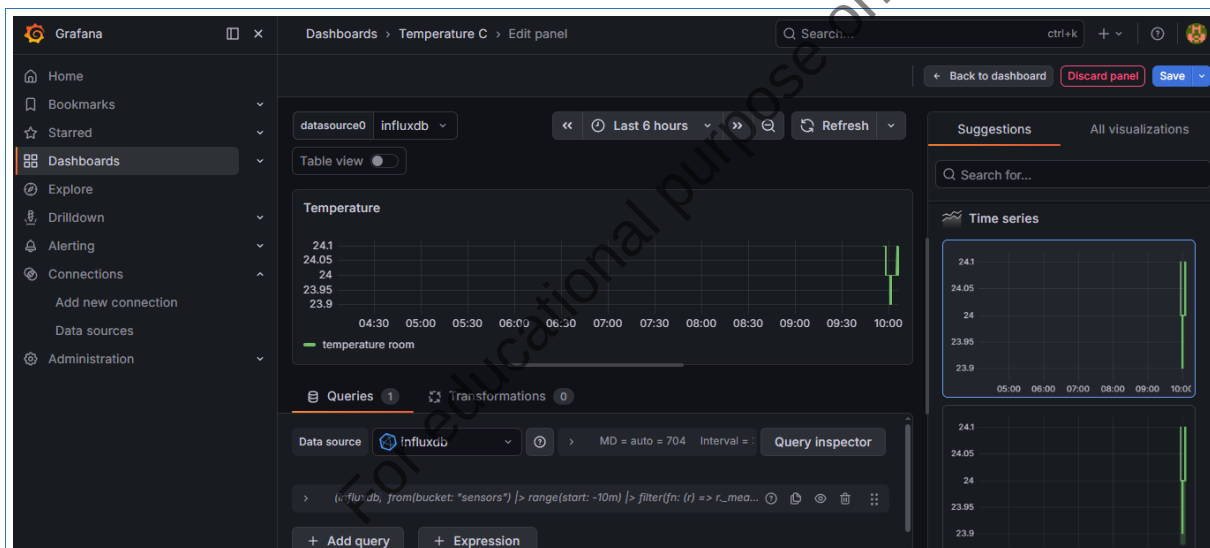


Figure 23 — First live temperature chart in Grafana — real sensor data from the room

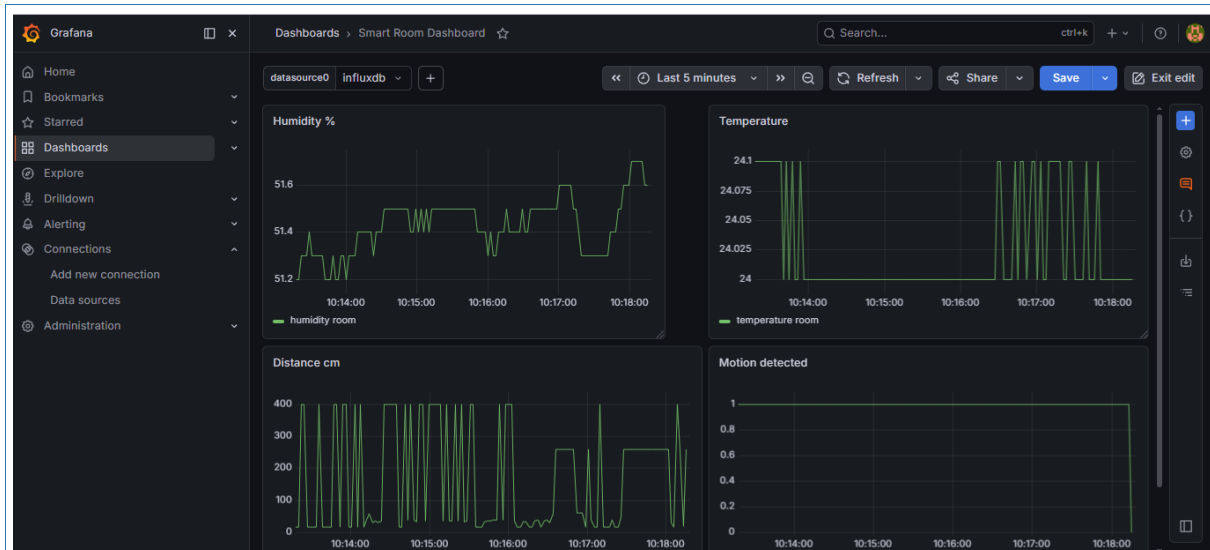


Figure 24 — Complete 4-panel Grafana dashboard: humidity, temperature, distance, and motion detection all live

6.5 Phase 4 — Web Dashboard

The Flask dashboard provides a unified interface combining live camera feed, sensor readings, system status, and alert history — all in one browser page accessible from any device on the WiFi network.

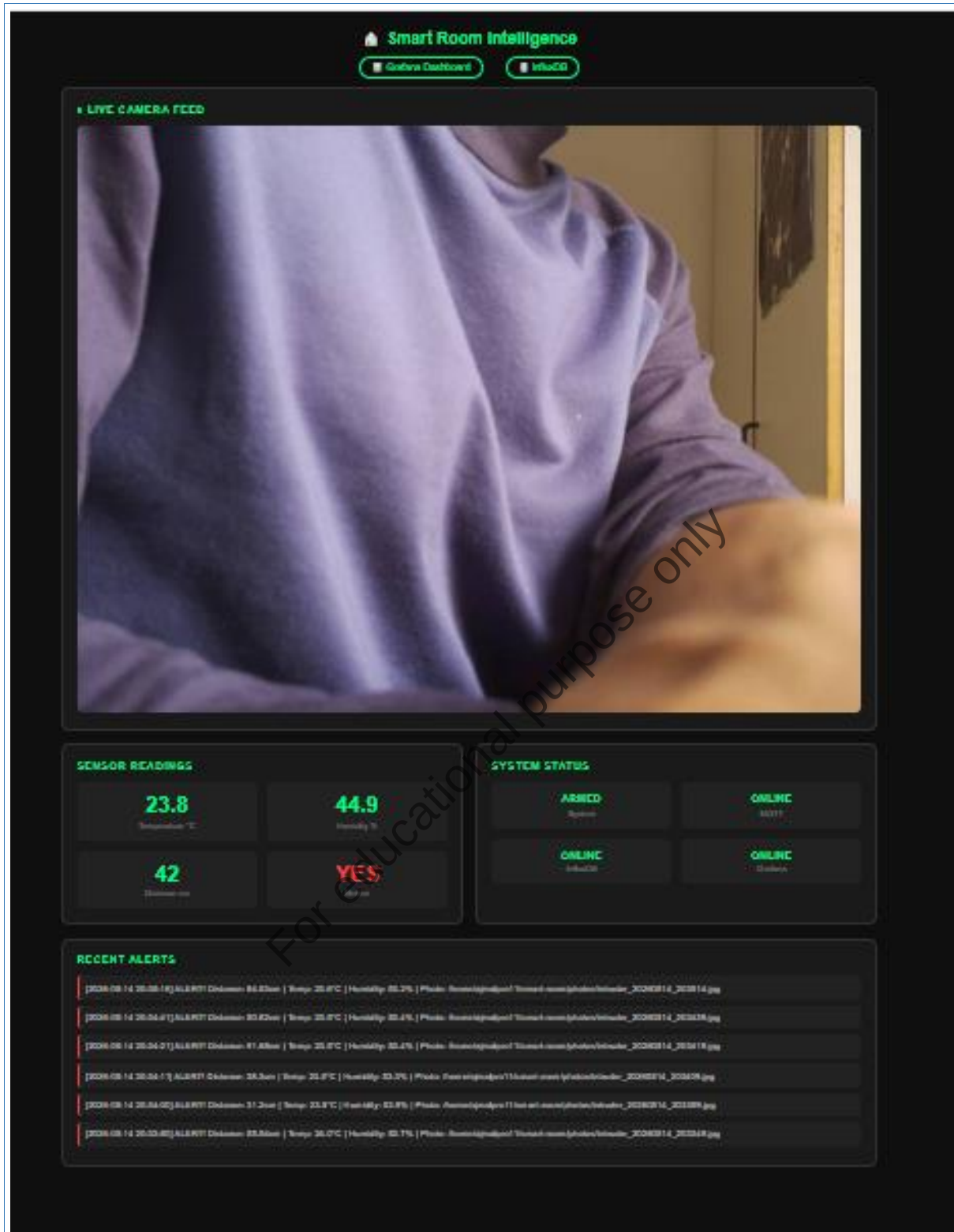


Figure 25 — Flask web dashboard running live: camera feed + sensor readings + system status + alert log

A critical architectural decision was separating camera frame capture from frame serving using a shared buffer with threading. Lock — this allows Flask to respond to /sensors API calls without being blocked by the continuous camera stream.

Remote access via Ngrok provides a secure HTTPS tunnel to the local server, with HTTP Basic Authentication protecting against unauthorised access. The free tier provides 8-hour sessions suitable for demonstrations.

For educational purpose only

7. Step-by-Step Replication Guide

This section provides a complete, tested guide for replicating the project from scratch. Every command is provided in exact form. All commands were tested on Raspberry Pi 5 running Debian GNU/Linux 13 (Trixie).

7.1 Hardware Wiring

Step 1 — Power off the Pi

```
| sudo shutdown -h now
```

Wait until the green LED stops blinking. Never wire sensors with the Pi powered on.

Step 2 — Wire HC-SR04

HC-SR04 Pin	Connection	Pi Pin
VCC	Direct	Pin 2 (5V)
GND	Direct	Pin 6 (GND)
TRIG	Direct	Pin 12 (GPIO 18)
ECHO	1kΩ → junction → Pi; 2kΩ junction to GND	Pin 18 (GPIO 24)

Step 3 — Wire IR Sensor

IR Pin	Connection	Pi Pin
+ (VCC)	Direct	Pin 1 (3.3V)
- (GND)	Direct	Pin 9 (GND)
OUT	Direct	Pin 16 (GPIO 23)

Step 4 — Wire DHT22

DHT22 Pin	Connection	Pi Pin
+ (VCC)	Direct	Pin 1 (3.3V)
- (GND)	Direct	Pin 9 (GND)
out (DATA)	Direct	Pin 11 (GPIO 17)

Step 5 — Wire Buzzer

Buzzer Pin	Connection	Pi Pin
+ (positive)	Direct	Pin 22 (GPIO 25)
- (negative)	Direct	Pin 20 (GND)

Step 6 — Connect Pi Camera

Lift CAM0 latch (slides outward on Pi 5). Insert ribbon cable with blue side facing away from USB ports. Press latch firmly. CAM0 is the connector closest to the USB ports.

7.2 System Setup

Step 7 — Update system

```
|sudo apt update && sudo apt upgrade -y
```

Step 8 — Enable I2C, SPI, 1-Wire

```
|sudo raspi-config
```

Navigate: Interface Options → enable I2C, SPI, and 1-Wire. Reboot when prompted.

Step 9 — Verify I2C and Camera

```
|sudo apt install -y i2c-tools
|sudo i2cdetect -y 1          # should show grid of dashes
|rpicom-hello --list-cameras # should show imx708
```

7.3 Project Setup

Step 10 — Clone repository

```
|git clone https://github.com/ajmalpro11/smart-room-intelligence.git
|cd smart-room-intelligence
```

Step 11 — Create virtual environment

```
|python3 -m venv venv --system-site-packages
|source venv/bin/activate
```

Step 12 — Install system dependencies

```
|sudo apt install -y liblgpio-dev swig libcap-dev
```

Step 13 — Install Python libraries

```
|pip install RPi.GPIO gpiozero smbus2 adafruit-circuitpython-dht
|pip install opencv-python pillow flask paho-mqtt requests
|pip install flask-socketio gevent gevent-websocket lgpio
|pip install influxdb-client python-dotenv picamera2
```

Step 14 — Set GPIO backend permanently


```
echo 'export GPIOZERO_PIN_FACTORY=lgpio' >> ~/.bashrc
source ~/.bashrc
```

Step 15 — Configure credentials

```
cp .env.example .env
nano .env      # fill in your passwords
```

7.4 Docker Setup

Step 16 — Install Docker

```
curl -fsSL https://get.docker.com -o get-docker.sh
sudo sh get-docker.sh
sudo usermod -aG docker $USER
newgrp docker
```

Step 17 — Fix Grafana permissions

```
sudo chown -R 472:472 ~/smart-room/grafana/data/
```

Step 18 — Start Docker stack

```
docker compose up -d
docker compose ps    # all three should show 'Up'
```

7.5 Sensor Testing

Step 19 — Test each sensor

```
source venv/bin/activate && export GPIOZERO_PIN_FACTORY=lgpio
python3 sensors/ultrasonic.py    # wave hand, see distance change
python3 sensors/ir_sensor.py    # wave hand, see MOTION DETECTED
python3 sensors/dht22.py        # see temperature and humidity
python3 sensors/camera.py       # press Enter, photo saved
```

7.6 Running the Full System

Step 20 — Start everything

```
~/smart-room/start.sh
```

Step 21 — Access dashboards

- Flask Dashboard: http://YOUR_PI_IP:5000
- Grafana: http://YOUR_PI_IP:3000 (admin / your_password)
- InfluxDB: http://YOUR_PI_IP:8086 (admin / your_password)

Step 22 — Run intruder detection

```
|python3 processor/decision_engine.py
```

Step 23 — Stop everything

```
|~/smart-room/stop.sh
```

Step 24 — Remote access via Ngrok (optional)

```
|ngrok config add-authtoken YOUR_TOKEN  
|ngrok start smart-room
```

Share the generated HTTPS URL and your configured username/password with collaborators.

For educational purpose only

8. Challenges and Solutions

The following table documents every significant challenge encountered during development, along with its root cause and solution. This section is particularly valuable for anyone replicating the project on Raspberry Pi 5 with Debian Trixie.

Challenge	Root Cause	Solution
RPi.GPIO fails on Pi 5	Pi 5 uses RP1 chip, not BCM2835	Switch to gpiozero + lgpio backend
lgpio pip install fails	Missing liblgpio-dev and swig	<code>sudo apt install liblgpio-dev swig -y</code>
libcamera not in venv	System-level C bindings	Create venv with <code>--system-site-packages</code>
Grafana permission denied	Container runs as UID 472, folder owned by root	<code>sudo chown -R 472:472 grafana/data/</code>
Camera not detected	Ribbon cable blue side wrong direction	Blue side faces away from USB ports on Pi 5
MPU-6050 not on I2C bus	Header pins not soldered to PCB	Pending university lab soldering session
Flask /sensors endpoint hangs	Camera generator blocking Flask	Threaded frame capture into shared buffer
Git push rejected	Remote had newer commits	<code>git pull --no-rebase</code> before every push
Docker Compose conflict	apt version conflicted with docker-ce	Use docker compose plugin not docker-compose v1
Ngrok config error	Version key format changed in v3	Run ngrok config upgrade
IR always triggered	Potentiometer miscalibrated	Slowly adjust until detection LED responds correctly
Sensor readings empty in Flask	gevent blocking MQTT thread	Switch SocketIO <code>async_mode</code> to <code>threading</code>
DHT22 constant RuntimeError	Reading too fast	Ensure <code>time.sleep(2)</code> between all reads

9. Results and Discussion

9.1 System Performance

Metric	Value	Notes
Sensor publish rate	Every 2 seconds	Limited by DHT22 hardware
Alert response time	< 0.5 seconds	From detection to buzzer beep
Camera stream frame rate	~15-20 fps	640x480 at JPEG quality 70
Still capture resolution	1920 x 1080	Full HD alert photos
Dashboard refresh	Every 2 seconds	HTTP polling
MQTT payload size	~120 bytes	JSON per reading
Data points per day	~172,800	4 fields × 43,200 readings
System RAM usage	~800 MB	All services running simultaneously

9.2 Sensor Accuracy

Sensor	Measured Accuracy	Datasheet Specification
HC-SR04 Distance	±1-3cm in testing	±3mm specification
DHT22 Temperature	Within 0.5°C of reference thermometer	±0.5°C specification
DHT22 Humidity	Within 3% of reference hygrometer	±2-5% specification
IR Detection Range	Consistent 4-25cm with calibration	2-30cm specification

9.3 System Reliability

- DHT22 RuntimeError handling prevents sensor errors from crashing publisher
- Alert cooldown prevents log flooding from sustained presence in detection zone
- Docker restart: unless-stopped policy ensures services auto-recover from crashes
- Threading lock on camera buffer prevents race conditions
- MQTT retain/persistence in Mosquitto ensures messages survive broker restarts

9.4 Security Implementation

- Credentials stored in .env file, excluded from Git via .gitignore
- .env.example provides template without exposing actual values
- Ngrok Basic Authentication prevents unauthorised public access
- All services local-only by default — public exposure requires explicit Ngrok setup
- InfluxDB and Grafana use non-default passwords via environment variables

For educational purpose only

10. Future Work

Enhancement	Description	Complexity
MPU-6050 Integration	Solder header pins in university lab, add vibration/tamper detection to pipeline	Low
ML Anomaly Detection	Train scikit-learn Isolation Forest on InfluxDB history to flag unusual sensor patterns	Medium
Boot Auto-Start	systemd service file to start all components automatically on Pi power-on	Low
Telegram Alerts	Send intruder photo via Telegram Bot API to phone immediately on detection	Medium
Pan-Tilt Camera	28BYJ-48 stepper motor mount to track detected persons across the frame	Medium
Person Detection	YOLOv8 or TensorFlow Lite to distinguish humans from other motion sources	High
Multi-Room	Multiple Pi nodes publishing to same MQTT broker with location tags	Medium
HTTPS Local	Self-signed certificate for encrypted local dashboard access	Medium
Permanent URL	Cloudflare Tunnel (free) for stable public URL without Ngrok session limits	Low
Mobile App	React Native app consuming the Flask API for native mobile experience	High

11. Conclusion

The Smart Room Intelligence System successfully demonstrates that a complete, professional-grade IoT security and monitoring platform can be built on a single-board computer using entirely open-source tools at minimal cost. The project integrates hardware interfacing, real-time data processing, IoT messaging, time-series storage, data visualisation, and web development into a cohesive, fully-documented system.

The primary technical challenge — Raspberry Pi 5 compatibility — required switching from the legacy RPi.GPIO library to the newer gpiozero and lgpio stack. This is thoroughly documented to save future developers the significant debugging time it initially required.

The modular architecture makes the system straightforward to extend. Each component — sensor scripts, MQTT publisher/subscriber, decision engine, Flask dashboard — is independently testable and replaceable. New sensors or features can be added without disrupting existing functionality.

From an educational perspective, this project provides hands-on experience across hardware, embedded programming, networking, databases, containerisation, and web development simultaneously — a combination that few single projects can offer. It serves as an ideal capstone project for students in mechatronics, computer science, or electrical engineering.

All source code, configuration files, and 27 development progress screenshots are available at github.com/ajmalpro11/smart-room-intelligence, enabling complete reproduction of the project by anyone following the step-by-step guide in Section 7.

Appendix A — GPIO Pin Reference

Pin 1 is top-left with Pi USB ports facing down. Left column = odd pins, right column = even pins.

Physical Pin	GPIO	Function	Used For
Pin 1	—	3.3V Power	IR sensor VCC, DHT22 VCC
Pin 2	—	5V Power	HC-SR04 VCC
Pin 3	GPIO 2 (SDA)	I2C Data	MPU-6050 SDA (pending)
Pin 5	GPIO 3 (SCL)	I2C Clock	MPU-6050 SCL (pending)
Pin 6	—	GND	HC-SR04 GND
Pin 9	—	GND	IR sensor GND, DHT22 GND
Pin 11	GPIO 17	General I/O	DHT22 DATA
Pin 12	GPIO 18	General I/O	HC-SR04 TRIG
Pin 16	GPIO 23	General I/O	IR sensor OUT
Pin 18	GPIO 24	General I/O	HC-SR04 ECHO (via voltage divider)
Pin 20	—	GND	Buzzer GND
Pin 22	GPIO 25	General I/O	Buzzer signal

Appendix B — Complete Library Reference

Library	Version	Purpose
gpiozero	2.0.1	High-level GPIO control (Pi 5 compatible)
lgpio	0.2.2.0	Pi 5 GPIO backend driver
smbus2	0.6.1	I2C protocol communication
adafruit-circuitpython-dht	4.0.12	DHT22 temperature/humidity sensor
picamera2	0.3.36	Pi Camera Module 3 control
opencv-python	4.13.0	Image processing and JPEG encoding
Pillow	12.2.0	Image file handling
Flask	3.1.3	Web server framework
flask-socketio	latest	WebSocket support
gevent	latest	Async networking
paho-mqtt	2.1.0	MQTT protocol client
influxdb-client	1.50.0	InfluxDB 2.x Python client
python-dotenv	latest	Secure environment variable loading
requests	2.33.1	HTTP client
numpy	2.4.4	Numerical processing

System Dependencies

Package	Install Command	Required For
liblgpio-dev	sudo apt install liblgpio-dev	lgpio compilation
swig	sudo apt install swig	lgpio C binding generation
libcap-dev	sudo apt install libcap-dev	picamera2 build
i2c-tools	sudo apt install i2c-tools	I2C debugging
docker-ce	curl -fsSL https://get.docker.com sh	Container runtime
ngrok	sudo apt install ngrok	Remote tunnel (optional)

Appendix C — Docker Services Reference

Parameter	Mosquitto	InfluxDB	Grafana
Image	eclipse-mosquitto:latest	influxdb:2.7	grafana/grafana:latest
Port	1883	8086	3000
Default user	anonymous	admin	admin
Password	none required	INFLUXDB_PASSWORD env	GRAFANA_PASSWORD env
Data persistence	mosquitto/data/	influxdb/data/	grafana/data/
Config location	mosquitto/config/mosquitto.conf	env variables	grafana/dashboards/

For educational purpose only

Appendix D — Flask API Endpoints

Endpoint	Method	Response Type	Description
/	GET	HTML	Main dashboard page
/video_feed	GET	multipart/x-mixed-replace	Continuous MJPEG camera stream
/sensors	GET	JSON object	Current sensor readings
/alerts	GET	JSON array	Last 10 alert log entries

Example /sensors JSON Response

```
{
  "distance": 45.23,
  "humidity": 51.4,
  "motion": false,
  "temperature": 23.5
}
```

For educational purpose only

Appendix E — Troubleshooting Guide

Symptom	Likely Cause	Fix
Cannot determine SOC peripheral base address	RPi.GPIO on Pi 5	<code>export GPIOZERO_PIN_FACTORY=lgpio</code>
No module named lgpio	lgpio not installed	<code>apt install liblgpio-dev swig; pip install lgpio</code>
GPIO busy error	Another script using pins	<code>pkill -f python3</code> , then restart
No cameras available	Ribbon cable orientation	Blue side faces away from USB ports
No module named libcamera	venv missing system packages	Recreate with <code>--system-site-packages</code>
Grafana permission denied	Wrong folder ownership	<code>sudo chown -R 472:472 grafana/data/</code>
Flask sensor values empty	Publisher not running	Restart <code>start.sh</code> , check all processes
MPU-6050 not detected	Pins not soldered	Solder header pins
DHT22 constant RuntimeError	Reading too fast	Ensure <code>time.sleep(2)</code> between reads
MQTT not connecting	Container not running	<code>docker compose up -d</code>
Git push rejected	Remote has newer commits	<code>git pull --no-rebase</code> , then push
IR sensor always YES/NO	Potentiometer miscalibrated	Slowly adjust until detection LED responds
<code>start.sh</code> not found	Alias not loaded	<code>source ~/.bashrc</code> or use full path <code>~/smart-room/start.sh</code>

— End of Report —